

Sécurité : surface d'attaque, IDOR et token de téléchargement

`SECURITY.md` découpe la sécurité de DataShare en **zones de menace** (authentification, accès fichier, upload, injection, fuite d'information, transport, secrets), avec pour chacune la menace et le traitement. Cette fiche approfondit trois zones et deux choix qu'un jury peut interroger : pourquoi un token de téléchargement aléatoire de 192 bits, et pourquoi le fichier d'autrui répond **404** et non 403.

Authentification : JWT et mots de passe

- Mots de passe de compte hachés en **BCrypt** (coût par défaut), 8 caractères minimum.
- JWT (JSON Web Token) signé **HS256** (HMAC, clé symétrique ≥ 32 octets lue dans l'environnement en prod), émis via `NimbusJwtEncoder`, validé par `oauth2-resource-server` (`exp`, `iss`) : pas de code de sécurité maison.
- Access token de **1 h**, pas de refresh token (re-login) : simplicité assumée pour le MVP (Minimum Viable Product).
- Connexion : message **identique** pour email inconnu et mot de passe faux (`401 INVALID_CREDENTIALS`), la vraie cause n'apparaît que dans le champ `debug` (mode verbeux). Cela évite l'énumération de comptes.
- `GET /api/me` relit le compte en base : un token valide dont le compte a disparu est rejeté.

Accès fichier : deux menaces distinctes

Menace	Question de l'attaquant	Défense
Token devinable	« Puis-je deviner le lien d'un fichier ? »	token <code>SecureRandom</code> 192 bits, non séquentiel
IDOR (Insecure Direct Object Reference)	« Puis-je accéder à <code>/api/files/<id></code> d'un autre en changeant l'id ? »	<code>owner_id</code> filtré dans la requête, 404 sinon

Pourquoi 192 bits, aléatoire, non séquentiel

```
private static final SecureRandom RANDOM = new SecureRandom();
private static final int BYTES = 24;
...
RANDOM.nextBytes(buffer);
return Base64.getUrlEncoder().withoutPadding().encodeToString(buffer);
```

- 24 octets = 192 bits d'entropie ; en base64url, 32 caractères utilisables dans une URL.
- `SecureRandom` est un générateur cryptographiquement sûr ; `java.util.Random` est prédictible après quelques sorties.
- Non séquentiel** : avec des ids 1, 2, 3, connaître un lien donne les voisins. Un aléa de 192 bits rend l'énumération infaisable en pratique.
- Colonne `UNIQUE` en base : filet contre une collision, jamais attendue.
- Le lien de partage est une **capacité** : quiconque le possède peut télécharger. Il doit donc être impossible à deviner. L'option mot de passe fichier s'ajoute par-dessus.

Pourquoi 404 et pas 403

```
StoredFile file = files.findByIdAndOwnerId(id, ownerId)
    .orElseThrow(() -> new ResourceNotFoundException("Fichier introuvable", "id inconnu : " + id));
```

La requête ne charge le fichier que s'il appartient à l'appelant : pour un fichier d'autrui, le résultat est vide, exactement comme pour un id qui n'existe pas. Le code renvoie donc un **404** identique dans les deux cas.

Réponse	Ce qu'elle révèle à l'attaquant
403 Forbidden	« ce fichier existe, mais il n'est pas à toi » : confirme l'existence de l'id
404 Not Found	rien : id inconnu et id d'autrui sont indiscernables

Le filtre dans la requête (et non un `if (file.owner != caller)` après chargement) rend l'oubli de contrôle impossible : le fichier d'autrui n'est jamais chargé. L' `owner_id` vient du **sujet du JWT**, jamais d'un paramètre du client.

Upload

- Limite **1 Go** (Spring et `client_max_body_size` nginx) : évite le DoS (Denial of Service) disque.
- **Liste noire d'extensions** (`exe`, `bat`, `sh`, `jar` ..., configurable) **et** détection du type réel par octets (`tika-core`) : rattrape un exécutable renommé. Le `Content-Type` servi vient de la détection, jamais du client.
- Limite connue et documentée : `tika-core` ne trahit pas `.msi`, `.jar`, ni les scripts texte (`ps1`, `vbs`, `js` ...); pour eux seule la liste noire par nom protège.
- Nom d'objet de stockage généré (`UUID.randomUUID()`), jamais le nom fourni : pas de path traversal. Le nom affiché est assaini (composants de chemin retirés).
- Upload **streamé** (seuil 2 Mo puis disque temporaire) : évite l'OOM (Out Of Memory) sur 1 Go.

Analyse et suivi

SECURITY.md distingue ce qui est fait (☑), à faire (□) et en attente (⌚). Outils : `npm audit`, gitleaks, Trivy (non bloquant, image `si10` hors prod), CodeQL (SAST, Static Application Security Testing), Dependabot. Un run manuel OWASP (Open Worldwide Application Security Project) dependency-check a remonté 2 CVE (Common Vulnerabilities and Exposures) ≥ 7 sur le back, notées **non encore traitées**. Hors MVP : antivirus, rate-limiting, 2FA, rotation de clé JWT, audit.

Dans DataShare

- Toutes les décisions : `SECURITY.md` (§1 surface, §2 décisions).
- Token : `DownloadTokens.java`.
- 404 : `StoredFileRepository.java:15` et `FileService.java:155-158`.
- Contrôles d'upload : `FileService.upload`.
- Deux chaînes de filtres : `SecurityConfig.java`.

À retenir

1. Le lien de partage est une capacité : 192 bits `SecureRandom`, non séquentiel, donc impossible à deviner ou à énumérer.
2. IDOR = changer un id pour atteindre la ressource d'autrui. Parade : filtrer par `owner_id` (sujet du JWT) **dans la requête**.

3. 404 plutôt que 403 : on ne révèle pas l'existence des ressources d'autrui.
4. Upload : taille limitée, liste noire d'extensions + type réel par octets, nom de stockage généré ; limites de `tika-core` documentées.
5. SECURITY.md sépare honnêtement fait, à faire et hors MVP (refresh token, rate-limiting, antivirus, CVE back en attente).